Lab2:BombLab 实验报告

梁心一 25800190015

一、以 tutor 为例的分析流程

1.实验目的与分析思路

本次实验要求从 bomb 的反汇编代码进行静态分析，推断出每一个问题的内部运行逻辑，并攥写等效的 C 程序 defuser.c。对每个程序，由随机的 seed 生成 challenge，作为函数的计算输入。在 tutor 中，为 phase_00，目的是利用上述 defuser.c 计算得到口令，来解出 bomb。

在分析中，发现除了函数本身，函数还会调用辅助函数，这需要根据进一步的反汇编去分析辅助函数的作用。整体分析流程是

1）先确认函数入口参数和输入格式，再沿着汇编的数据流分析出需要处理的目标和各种运算。

2）得到当前 seed，challenge 的值，推断出中间值并与汇编断点调试中的实际值比较，确认推断无误之后，把算出口令交给 test 程序验证。

2.观察入口参数与解析结果

1）输入的初步检查

由于不知道正确口令，先输入“0 0”，观察程序怎样处理这两个数。

输入后，用 disassemble /r phase_00 查看函数的反汇编，反汇编给出的入口地址是 0x401480，因此在此设置断点。

暂停后，先执行 x/i $pc，确认程序停在哪里，结果显示当前指令是 endbr64，与反汇编中的函数入口一致，说明断点位置正确。

结合 tutor 给出的函数原型和课程介绍的 Linux x86-64 参数传递约定，函数的前两个参数分别通过 rdi 和 rsi 传入。执行 x/s $rdi。输出为“0 0”，证明该寄存器就是存储输入函数的原始字符串。

第二个参数是 challenge 数值，所以执行 p/x $rsi。0xbdd732262feb6e95，与本次运行的 challenge 一致，因此可以确认入口处 rsi 保存的是本关的 challenge。

2）输入格式的判断

继续查看反汇编中的 sscanf 调用，并在它返回后的 0x4014e7 处暂停。sscanf 的返回值表示成功转换并赋值的字段数，可以从 eax 中观察。本次 eax 为 2，说明两个字段都已成功转换。

为了确认转换后的整数存放在哪里，我对照调用 sscanf 前准备的目标地址，找到 rbp-0x20 和 rbp-0x1c 两个位置。

这两个地址相差 4 字节，与本关每个输入整数占用的空间一致。执行 x/2uw ($rbp-0x20)，从第一个位置开始连续读取两个整数。于是基本可以确认“0 0”已经被正确解析。

tutor 中，已经从指导文件中知道了应该先输入两个无符号整数 0 0 来测试，但是之后的题目并不知道，于是，可以参照以上流程，根据对 sscanf 的解析，来得到输入的数量和格式要求。

3.分析第一个整数的比较流程

tutor 中已知函数关系。

在第一个比较点 0x4014fa，我沿着 challenge 的计算过程查看相关指令。程序先将 challenge 逻辑右移 5 位，保留结果的低 8 位，再与常量 0x2a 异或。因此，第一个目标整数 a 的计算关系为：

a = ((challenge >> 5) & 0xff) ^ 0x2a

本次 seed 为 42，GDB 中比较指令处的 edi 为 94，而输入的第一个整数仍为 0。将 challenge 值 0xbdd732262feb6e95 代入上式，结果也是 94，与寄存器中的读数相符。

继续单步执行比较和条件跳转后，程序进入 0x4014b0 处的失败路径，没有继续调用 tutorial_mix，也没有检查第二个整数。随后执行 finish 返回 main，观察到 eax 为 0，即 phase_00 返回 false。由此可以确定，这次失败发生在第一个整数的比较处，输入解析本身没有问题。

4.分析辅助函数并恢复第二个整数

查看 tutorial_mix 的反汇编，其中的关键指令是：

lea 0x7(%rdi,%rdi,2),%eax

这里的 lea 计算了 rdi + 2*rdi + 7。结果写入 eax，只保留低 32 位。phase_00 把第一个目标整数传给 tutorial_mix，再用返回值检查第二个输入，因此第二个整数 b 满足：

b = 3 * a + 7

至此，两个输入之间的关系已经明确：先由 challenge 求出 a，再计算 b。

5.计算与运行验证

本次 seed 为 42，challenge 为 0xbdd732262feb6e95。代入公式得到 a 为 94，b 为 289，因此输入为：

94 289

在终端运行./bomb --seed 42 --phase 0，输入上述口令后，程序输出：

Phase 0 defused.

程序成功通过，说明推导出的计算关系在本次运行中成立。其中，第一个目标值还与之前 GDB 里观察到的 94 一致。

实际解题中，基于 challenge 的计算并不已知，需要追踪 challenge 传入后的寄存器的值（以及之后与该值相关的一系列寄存器与指令），来反推。

二、正式解题记录

以下几题均固定 seed=42 来分析，通过之后再用 seed=1 检验，等 6 题全部通过后调用了 test 检验，最后再使用 test 里没有的 seed=43 来测试。

【第 1 题】

【注：篇幅问题第 1 题展示的较详细，后面的分析和调试过程略写文字】

1）查找 challenge 与输入格式要求

先执行 objdump -d --no-show-raw-insn --disassemble=phase_01 ./bomb，得到本题入口地址 0x402090。先找保存参数的指令

mov %rsi,%r12

mov %rdi,%rbx

由此知道后续应从 r12 追踪 challenge

传给 sscanf 的格式字符串地址为 0x403189，目标地址为 rbp-0x14，返回

后执行 cmp $0x1,%eax。结合该地址的格式字符串为“%u”，判断本题要求一个无符号十进制整数，结果保存在 rbp-0x14。

这里 eax 为 1 表示成功解析一个字段。

```
0000000000402090 <phase_01>:
  402090:       endbr64
```

Phase_01 起始地址

```
4020cb:       mov    %rbx,%rdi
4020ce:       call   4011e0 <__isoc99_sscanf@plt>
4020d3:       cmp    $0x1,%eax
```

sscanf 调用

```
(gdb) x/s 0x403189
0x403189:       "%u"
```

格式字符串

2）追踪 challenge 相关数据流，分析算法

phase_01 把 r12 中的 challenge 传给 p1_compute，因此继续查看该函数的反汇编。这里没有直接给出目标值，而是依次调用 p1_step1、p1_step0、p1_step2，顺序与函数名中的编号不同，需要按照实际调用顺序分析。

p1_compute 先复制 challenge，再分别右移 32 位和 16 位。为了方便记录，把使用到的部分记为：

lo = (uint32_t)challenge

hi = (uint32_t)(challenge >> 32)

mid = (uint32_t)(challenge >> 16)

其中 mid 保留右移后的低 32 位，对应原数第 16 到 47 位，不能只取 16 位。

第一次调用 p1_step1 前，edi 中是 hi，esi 对应 mid。关键指令为：

xor $0x86ee,%esi

lea (%rsi,%rdi,4),%eax

xor 先对 mid 异或，lea 再计算加法与乘法，因此得到 t1 = (mid ^ 0x86ee) + 4*hi。结果写入 eax，按低 32 位保存。

第二次调用前，edi 改为 lo，esi 改为 hi，t1 暂存在 ecx。p1_step0 中的关键指令为：

lea (%rdi,%rdi,2),%eax

add %esi,%eax

xor $0x40e7,%eax

对应 t0 = (3*lo + hi) ^ 0x40e7。

最后调用 p1_step2，传入 t0 和 t1：

lea 0x3750(%rsi),%eax

xor %edi,%eax

得到 answer = (t1 + 0x3750) ^ t0。

```
0000000000402050 <p1_compute>:
  402050:       endbr64
  402054:       push   %rbp
  402055:       mov    %rdi,%rdx
  402058:       mov    %rdi,%rax
  40205b:       shr    $0x20,%rax
  40205f:       mov    %rdx,%rsi
  402062:       mov    %eax,%edi
  402064:       shr    $0x10,%rsi
  402068:       mov    %rsp,%rbp
  40206b:       call   402030 <p1_step1>
  402070:       mov    %edi,%esi
  402072:       mov    %edx,%edi
  402074:       mov    %eax,%ecx
  402076:       call   402020 <p1_step0>
  40207b:       mov    %ecx,%esi
  40207d:       mov    %eax,%edi
  40207f:       call   402040 <p1_step2>
  402084:       pop    %rbp
  402085:       ret
```

p1_compute 对三个辅助函数的调用

```
0000000000402040 <p1_step2>:
  402040:       endbr64
  402044:       lea    0x3750(%rsi),%eax
  40204a:       xor    %edi,%eax
  40204c:       ret
```

p1_step2 对最终结果的计算

3）根据推导出来的逻辑动态比较

静态分析得到关系后，执行 gdb --args ./bomb --seed 42 --phase 1，在 0x4020e0 设置断点。这条指令位于 p1_compute 返回之后、最终比较之前，适合检查目标整数。运行时先输入 0，使程序完成解析并到达断点。

执行 p/x $r12，读到本题 challenge 为 0x28efe333b266f103。执行 p/u $eax，读到目标整数为 3335917183。

另按前面的公式计算，得到 lo=2993090819、hi=686809907、mid=3811816038，进一步得到 t1=2264056148、t0=1076164315，最终 answer=3335917183。实际断点直接观察的是最终 eax，两者结果一致。

```
Phase 1 challenge: 0x28efe333b266f103
Input (unsigned decimal integers):
0

Breakpoint 1, 0x00000000004020e0 in phase_01 ()
(gdb) p/x $r12
$1 = 0x28efe333b266f103
(gdb) p/u $eax
$2 = 3335917183
(gdb)
```

对 phase1 进行断点测试

退出 GDB 后，运行./bomb --seed 42 --phase 1，输入 3335917183，确认第 1 题解除

4）编写对应的 C 代码，并调用单题测试

这一题的程序只是根据 challenge 进行几次 32 位无符号数值计算：先取

challenge 的低 32 位、中间 32 位和高 32 位，分别经过异或、乘法、加法后，再组合成一个最终的 answer。

代码直接见文件

重新编译后测试：

./defuser --seed 42 --phase 1 | ./bomb --seed 42 --phase 1

./defuser --seed 1 --phase 1 | ./bomb --seed 1 --phase 1

两次均得到 Phase 1 defused. 。其中 seed=1 时 challenge 为 0xbeeb8da1658eec67，说明实现没有固定使用 seed=42 的口令。

```
Phase 1 challenge: 0x28efe333b266f103
Input (unsigned decimal integers):
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

第一关验证截图

【第 2 题】

1）先确定输入数量和数据位置

反汇编中先看到 phase_02 把一个格式字符串传给 sscanf，读取位置连续分布在 rbp-0x40 到 rbp-0x2c，之后又比较 sscanf 的返回值是否为 6。格式字符串为“%u %u %u %u %u %u”，所以这一题不是输入一个数，而是输入 6 个无符号整数。

```
(gdb) x/s 0x40317a
0x40317a:       "%u %u %u %u %u %u"
```

第 2 关输入分析

2）静态推断

继续看 sscanf 返回后的第一处比较，程序没有直接把 challenge 和某一个输入相比较，而是先把 challenge 右移 38 位，再用 and $0xf 保留 4 位，之后通过 lea 完成乘 3，最后和第一个输入比较。由此可以推断第一个输入是由 challenge 的一部分计算得到的，而不是任意常量。为了便于描述，将这第一个输入记为 a[0]，得到：

a[0]=3*((challenge>>38)&0xf)

3）从循环结构推断后续元素关系

首项比较通过后，程序跳到一个会重复执行的地址。每次循环都读取上一次的输入值，先进行两倍运算；同时根据循环计数改变移位量，从 challenge 中取出 5 位，再和前面的结果异或，最后加上一个随循环增加的数。

根据第一次循环中移位量为 4、加数为 11，之后两个量分别每轮增加 3 和 11，可以把当前元素记为 a[i]，把上一个输入记为 a[i-1]，得到：

a[i]=((2*a[i-1])^((challenge>>(3*i+1))&0x1f))+11*i，1≤i≤5

因此这题本质上是根据 challenge 生成一个长度为 6 的递推序列，再依次比较输入的 6 个数。所有中间结果在 32 位寄存器中计算，C 程序中使用 uint32_t 表示。

```
40215d:       mov    -0x40(%rbp),%edx
402160:       shr    $0x26,%rax
402164:       and    $0xf,%eax
402167:       lea    (%rax,%rax,2),%eax
40216a:       cmp    %eax,%edx
```

首项计算

```
40217f:       add    %edx,%edx
402181:       shr    %cl,%rax
402184:       and    $0x1f,%eax
402187:       xor    %edx,%eax
402189:       mov    0x4(%rsi),%edx
40218c:       add    %edi,%eax
40218e:       cmp    %eax,%edx
```

递推计算

4）动态对比

seed42 下 challenge 为 0x47526757130f9f52。第一次使用全 0 输入时，首项比较位置的 eax 为 39，而输入位置的 edx 为 0，说明程序确实先检查由 challenge 生成的首项。重新运行并把首项改为 39 后，程序进入循环比较，eax 为 102，栈上仍能看到后面的输入是 0。再把前两项改为 39 和 102 继续运行，程序在同一比较位置得到下一项目标 232，同时 ecx 为 7，说明循环计数和移位量已经发生变化。这个过程验证了“当前项依赖前一项，并且每轮使用 challenge 的不同位”的判断；后续完整序列再由同一递推式计算。

```
Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
0 0 0 0 0 0

Breakpoint 1, 0x000000000040216a in phase_02 ()
(gdb) p/x $r13
$1 = 0x47526757130f9f52
(gdb) p/u $edx
$2 = 0
(gdb) p/u $eax
$3 = 39
(gdb) x/6uw ($rbp-0x40)
0x7fffffffcb40: 0       0       0       0
0x7fffffffcb50: 0       0
(gdb)
```

第 1 断点分析

```
Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
39 102 0 0 0 0

Breakpoint 1, 0x000000000040216a in phase_02 ()
(gdb) continue
Continuing.

Breakpoint 2, 0x000000000040218e in phase_02 ()
(gdb) p/u $ecx
$4 = 4
(gdb) p/u $edx
$5 = 102
(gdb) p/u $eax
$6 = 102
(gdb)
```

第 2 断点分析

5）编写对应的 C 代码，并进行验证

这一题的 C 程序先计算 a[0]，再用前一项和 challenge 的不同位依次计算 a[1]到 a[5]。代码直接见文件，seed42 和其他 seed 的通关结果统一放在报告末尾。

【第 3 题】

篇幅原因，本题开始，不再截图静态分析和动态分析的图片

1）确定输入格式

phase_03 对 sscanf 的返回值要求为 2，格式字符串为“%u %u”，两个数分别写入 rbp-0x18 和 rbp-0x14。

观察后面的两次比较可以发现：第一次比较直接使用第一个栈上数值，第二次比较使用一个函数调用的结果和第二个栈上数值比较。因此两个输入的作用不同，但此时还只能确认第一个输入参与前置检查、第二个输入参与函数结果检查。

2）从数据流判断分支选择方式

第一次比较前，程序把 challenge 的低 3 位取出放入 edi，并与第一个输入比较。随后程序从 challenge 的不同位置取出两部分数据并异或，形成传给 p3_dispatch 的另一个 32 位参数。结合后面使用该低 3 位结果访问跳转表的过程，再把这两个值分别记为分支编号 first 和计算参数 x：

```
first=(uint32_t)challenge&7
x=(uint32_t)(challenge>>32)^(uint32_t)(challenge>>3)
```

在辅助函数 p3_dispatch 中，原来的分支编号被保存，x 被放入计算参数寄存器，接着通过`0x405820`处的地址表间接跳转。不同的编号会跳到不同的代码位置，所以可以判断这里是 switch 形式的分支，而不是普通的顺序计算。

3）推断各分支

逐个查看跳转表指向的代码后，可以看到每个分支都只对 x 进行乘法、异或、移位和加法，最后返回一个 32 位结果。整理得到：

```
first=0：5*x+0xaeb8
first=1：(4*x)^0xa0e7
first=2：3*x+0x9c31
first=3：5*x+0x914b
first=4：(x^0x9fea)+(x>>2)
first=5：(x^0x681a)+(x>>1)
first=6：(2*x)^0x1d21
first=7：5*x+0x164ae
```

跳转表的实际对应顺序为 0、1、2、3、4、5、6、7 分别跳到 case4、case2、case3、case6、case0、case5、case7、case1。

4）动态对比

seed42 下 challenge 为 0x581ce1ff0e4ae394。GDB 中读取 challenge 后，用表达式分别求出了低 3 位 4、混合值 x=3117792653，以及按 first=4 对应分支公式计算出的 3897201098。

随后输入“4 3897201098”确认第三题解除，说明分支编号、x 的计算和分支公式组合起来能够得到正确口令。C 程序用 switch 还原分支，代码直接见文件；通用测试结果统一放在报告末尾

【第 4 题】

1）输入推断与静态分析

反汇编显示 phase_04 只要求 sscanf 成功解析 1 个字段，格式字符串为“%u”，输入保存在 rbp-0x14。

调用 p4_walk 前，程序从 challenge 取出低 7 位放入 edi，同时把 esi 设为 0、edx 设为 127；另一个参数由 challenge 右移 7 位和右移 32 位后的结果异或得到。

p4_walk 先执行`edx-esi`，将差值右移 1 位，再加回 esi，所得结果与 edi 比较。比较相等时进入返回路径；edi 较小时把右边参数改为中点减 1，edi 较大时把左边参数改为中点加 1，随后再次调用 p4_walk。由于每次调用都会缩小两个参数之间的范围，并且函数会调用自身，所以程序是在 0 到 127 的范围内递归查找 challenge 低 7 位对应的数。

推断程序意图，用下列变量表示各个寄存器值，可以把中点计算写成：

```
target=(uint32_t)challenge&0x7f
low=0
high=127
mid=low+((high-low)>>1)
salt=(uint32_t)(challenge>>7)^(uint32_t)(challenge>>32)
```

递归返回时，子调用结果先乘 3，再与`mid+salt`异或；根据目标值位于中点左侧还是右侧，结果分别再异或 0x1f 或 0x6。目标值等于中点时，返回`(mid+0xdb)^salt`。

seed42 下 challenge 为 0x09bc585a244823f2，计算得到 target=114、salt=0xbdf4c81d，沿着递归调用逐层缩小范围后，最终结果为 1438853860。

2）动态对比与 C 代码

越过 p4_walk 调用后观察到 eax 为 1438853860，与递归计算结果相同；输入该值后第 4 题解除。C 程序先按照相同方式得到 target 和 salt，再递归模拟区间变化和返回值计算，代码直接见文件。

【第 5 题】

1）输入分析

phase_05 要求 sscanf 成功解析 2 个字段，格式字符串为“%u %u”，两个输入分别保存到 rbp-0x1c 和 rbp-0x18。进入 p5_traverse 后，函数既有返回值，又通过指针写回一个数；phase_05 随后分别把这两个结果和两个输入比较。因此本题的两个输入不是同一个计算序列，而是函数返回值和指针输出值。

2）静态分析

p5_traverse 先取 challenge 低 3 位，用这个值从 0x405b60 读取一个地址。读取结果随后被继续引用，并访问偏移 0、8 和 12 处的数据：偏移 0 处的数据再次作为地址使用，偏移 8 和 12 处的数据参与整数运算。因此 0x405b60 保存的是节点入口地址，节点的偏移 0、8、12 分别对应后继地址、一个标记值和一个权重值。

函数先用 challenge 右移 6 位和右移 32 位后的低 32 位异或，再与 0xcd6c 异或得到累计值。处理节点时，累计值乘 3 后加偏移 12 处的权重值，再与偏移 8 处的标记值异或；标记值同时通过第二个参数指针写回。循环读取偏移 0 处的后继地址，并按照 challenge 的第 3、4 位决定循环次数，所以程序会沿节点的后继地址继续处理多个节点。

对应计算关系为：

```
acc=((uint32_t)(challenge>>6)^(uint32_t)(challenge>>32))^0xcd6c
acc=(3*acc+weight)^tag
```

起始编号=(uint32_t)challenge&7

移动次数=(uint32_t)(challenge>>3)&3

从起始节点开始，程序处理移动次数加 1 个节点；最终 acc 与第一个输入比较，最后处理节点的 tag 与第二个输入比较。

3）动态对比与 C 代码

seed42 下起始编号为 6、移动次数为 0，实际单步越过 p5_traverse 后读到返回值 2753921204，栈上输出位置读到 2950，两项都与公式计算一致。这个 seed 没有进入 next 移动循环。另用 seed=2 进行测试，按 challenge 得到起始编号 3、移动次数 2，程序成功通过，说明求解代码确实处理了两次 next 指针移动，而不是只适用于起始节点。C 程序代码直接见文件。

【第 6 题】

1）输入分析

phase_06 的格式字符串为“%u”，只解析一个无符号整数，保存到 rbp-0x14。phase_06 把 challenge 传给 p6_dispatch，再将其返回值和输入比较。

p6_dispatch 中连续出现两次通过 0x405e00 进行的间接 call。第一次用 challenge 低 3 位，按`0x405e00+索引*8`读取地址后调用；读取到的内容是代码地址。第二次使用 challenge 第 3 至 5 位，前面的指令已将该索引左移 3 位形成表内字节偏移，所以直接执行`call *0x405e00(%rbx)`。同一地址区域被当作调用目标使用，说明 0x405e00 保存的是函数地址表。

2）静态分析

第一次调用前，程序把 challenge 右移 6 位后的低 32 位与 challenge 高 32 位异或，结果放入第一次调用的参数寄存器；同时，challenge 低 3 位生成第一次表索引，第 3 至 5 位生成第二次表索引：

```
x=(uint32_t)(challenge>>6)^(uint32_t)(challenge>>32)
i=(uint32_t)challenge&7
j=((uint32_t)challenge>>3)&7
```

第一次 call 返回后，返回值立即与 0xfe90 异或，结果成为第二次 call 的参数；第二次返回值再加 0x35d0，最后与 phase_06 的输入比较。因此两次调用的完整数据流为：

```
t=F[i](x)
answer=F[j](t^0xfe90)+0x35d0
```

沿着 0x405e00 中的地址继续查看各个目标函数，可得到表项和运算的对应关系：0→op_1，1→op_4，2→op_4，3→op_2，4→op_4，5→op_5，6→op_3，7→op_0。各函数的主要运算分别为：

```
op_0(v)=((v<<7)|(v>>25))^0x1135
```

op_1(v)=(v^(v>>3))+0xc05b
op_2(v)=(v^0x3827)+8*v
op_3(v)=(v>>2)|(v<<30)
op_4(v)=(v+0x9384)^(v>>7)
op_5(v)=(4*v+0x4112)^(5*v)

其中 op_0 和 op_3 中的移位需要按循环移位理解，不能当作普通移位。

3）动态对比与 C 代码

seed42 下 challenge 为 0x37e9671c45376d5d，第一次调用返回 2030276347，异或常量后的第二次输入为 2030268523，第二次调用返回 1092537252，最后得到 1092551028，与动态比较目标一致。C 程序还原函数表和六个运算函数，代码直接见文件。

【第 7 题 Honor】

1）先从入口检查确定输入布局

Honor 不是直接读取三个独立整数。secret_gate 先复制整段输入，再检查输入长度和固定位置的 4 个字节。长度检查先减去 0x24，再和 0x7f 比较，说明总长度至少要达到 36 字节，并且超过 36 的部分不能超过 127 字节，即长度范围为 36 至 163 字节。偏移 32 处和 0x4e45504f 比较，按小端序看内存字节是 4f 50 45 4e，对应字符串“OPEN”。

因此输入的前 32 字节可以看作前缀，第 33 至 36 字节必须是 OPEN，后面还要放 secret_phase 需要解析的三个无符号整数。

secret_phase 使用格式“%u %u %u”，三个数分别保存到 rbp-0x2c、rbp-0x28 和 rbp-0x24。

```
40275c:       call   401190 <strlen@plt>
402761:       sub    $0x24,%rax
402765:       cmp    $0x7f,%rax
```

长度检查

```
402786:       call   402700 <secret_copy>
40278b:       cmpl   $0x4e45504f,-0xb0(%rbp)
```

偏移与比较

2）静态分析

secret_gate 调用 secret_phase 时，rdi 指向复制后的输入缓冲区，rsi 来自 phase_challenge，rdx 来自 transcript_context。secret_phase 随后分别使用 rsi 和 rdx 的 64 位内容：取各自高、低 32 位并异或，rdx 的低两位参与第一次比较。

按照这些寄存器数据流写成计算式：

a=(uint32_t)A^(uint32_t)(A>>32)
b=(uint32_t)B^(uint32_t)(B>>32)
first=(uint32_t)B&3

其中 A 对应 rsi 传入的 challenge，B 对应 rdx 传入的 context。接着，B 右移 16 位的结果与 first 乘以 0x8e42 异或；a 和 b 异或后进入 secret_mix，函数中的`ror $0x7`把 32 位数据循环右移 7 位，再与 0x364a 异或。第二次比较对应的计算为：

v=a^b

mask=(first*0x8e42)^(uint32_t)(B>>16)

second=(((v>>7)|(v<<25))^0x364a)^mask

第三次比较前，程序将 a 右移 7 位，与 b、first 和 0xb808 相加，再与 second 异或：

third=second^(b+first+0xb808+(a>>7))

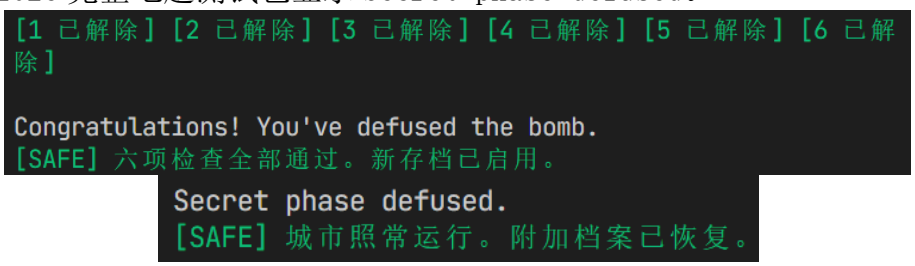
因此三个输入依次由 B 低两位、上面的 second 计算式和 third 计算式得到，且所有中间移位和运算按 32 位进行。

3）动态对比与 C 代码

公开测试和 seed43 完整测试均显示前缀、OPEN 标记和三个整数能够通过 secret_gate 及 secret_phase。C 程序按照上述输入布局生成前缀、OPEN 和三个整数，代码直接见文件。

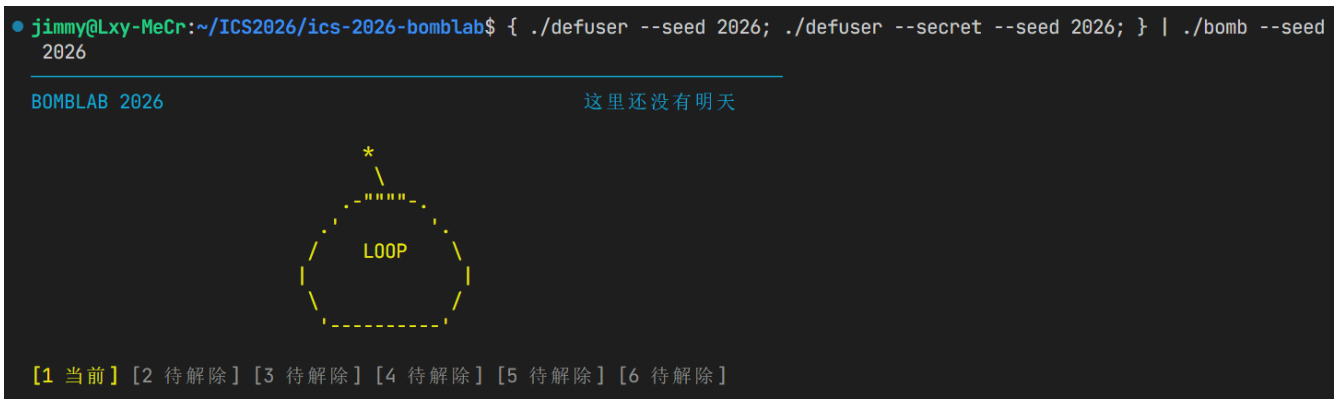
三、测试结果展示

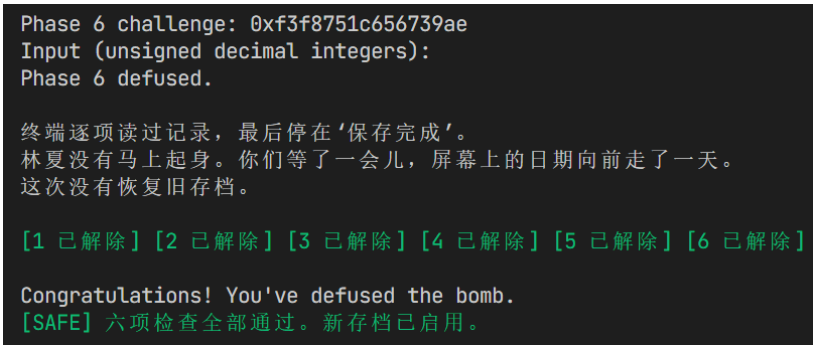
前六题公开测试结果为 60/60，Honor 公开测试结果为 10/10；公开列表外的 seed=2026 完整七题测试也显示 Secret phase defused。



6 小题与 Secret phase 检查通过

使用公开列表外 seed=2026 进行测试：





使用 seed=2026 的测试结果

```
jimmy@Lxy-MeCr:~/ICS2026/ics-2026-bomblab$ make test
python3 public_test.py
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
jimmy@Lxy-MeCr:~/ICS2026/ics-2026-bomblab$
```

完整测试结果展示

四、参考资料

1. https://cs4-challenge-lab-notes.readthedocs.io/zh-cn/latest/cmu_15213/lab2-bomblab.html#x01
2. https://byu-cpe.github.io/ecen224/programming-assignments/bomb-lab/

使用 AI 工具对汇编代码进行辅助分析